

AdaVerify: Adaptive Multi-Level Verification for Blockchain-Based AI Inference

Xiuze Dong¹, Zhao Wang^{1*}, Maosheng He¹, Wenxin Chen², Zhiqiang Liang¹,
Runze Zhou¹, and Qian Cheng¹

¹ Beijing Electronic Science and Technology Institute, Beijing, China
zhaowang.cs@gmail.com

² Shanghai University, Shanghai, China

Abstract. Deploying AI inference on blockchain demands verification mechanisms that balance cost, latency, and security. Existing approaches apply a uniform verification strategy to every request regardless of its risk profile, leading to either excessive gas costs or inadequate security guarantees. We present **AdaVerify**, an adaptive multi-level verification framework that dynamically selects among four verification levels—trust, multi-node redundancy, optimistic challenge, and zero-knowledge proof—based on a cost-optimal decision that incorporates stake value, provider-specific attack probabilities (derived from historical error rates), and gas price. We implement AdaVerify as a set of Solidity smart contracts on Ethereum and integrate EZKL for zero-knowledge proof generation over a LeNet-5 neural network. We evaluate AdaVerify through simulation across 8 scenarios and 20 statistical trials, using *measured gas costs* from Hardhat—including a real EZKL-generated Halo2 Solidity verifier (1.50M gas per proof verification)—and *real ZK proof benchmarks*. Results show that AdaVerify’s cost-optimal strategy reduces simulated net cost by 69.9% compared to the fixed-ZK baseline while achieving 98.7% attack detection—significantly higher than the fixed-L2 baseline (97.3%)—by selectively routing high-risk requests to ZK verification based on provider-specific risk. AdaVerify automatically adapts its level distribution in response to gas price (10–500 gwei), attack probability (5× parameter sweep), and alternative ZK backends (Groth16 vs. Halo2).

Keywords: Blockchain · AI Inference Verification · Zero-Knowledge Proofs · Adaptive Security · Smart Contracts

1 Introduction

Decentralized AI inference markets allow providers to serve model predictions on-chain, but a fundamental challenge remains: *how can a requester trust that the provider executed the correct model?* [2,3] Existing approaches apply a uniform verification strategy—either expensive zkML proofs for every request [1,2] or optimistic challenge-based protocols [3,4]—without adapting to per-request risk. A \$0.01 prediction and a \$10,000 financial decision receive identical treatment.

* Corresponding author

We propose **AdaVerify**, an adaptive framework with four verification levels—L0 Trust, L1 Redundancy, L2 Optimistic, L3 ZK Proof—selected per request by a cost-optimal strategy that minimizes gas cost plus expected attack loss, using provider-specific attack probabilities derived from historical error rates. Our contributions: (1) an adaptive verification framework with a closed-form cost-optimal selector (Eq. 2); (2) a full Solidity implementation with measured gas costs and EZKL-based Halo2 ZK proof integration; (3) evaluation across 8 scenarios, 20 trials, and 9 strategies, showing 69.9% net cost reduction vs. fixed-ZK with 98.7% detection, robust to gas price, ZK backend, attack model, and workload distribution.

2 Related Work

Verifiable AI inference. zkML frameworks [1,2,5,12] generate cryptographic proofs for neural network inference using Halo2 [15]/PLONK [16] circuits, but incur high proving costs (seconds to minutes per inference). Optimistic approaches [3,4,13] amortize cost via challenge windows but introduce latency and rely on rational verifiers. TEE-based methods [6,17] offer hardware-rooted trust but require specific assumptions. Platforms such as Ritual [7], Giza [8], veriML [11], and SafeML [14] provide on-chain AI inference but employ a single, fixed verification mechanism (Table 1). To our knowledge, no existing system adaptively selects verification intensity based on per-request risk.

Table 1. Feature comparison with existing on-chain AI verification systems. AdaVerify is the only framework supporting adaptive multi-level verification with per-request risk-based selection.

System	ZK Proof	Optimistic	Redundancy	Adaptive	Risk-based
EZKL [1]	✓	–	–	–	–
opML [3]	–	✓	–	–	–
Giza [8]	✓	–	–	–	–
Ritual [7]	–	–	✓	–	–
SafeML [14]	–	–	✓	–	–
veriML [11]	–	–	✓	–	–
AdaVerify	✓	✓	✓	✓	✓

Adaptive security and mechanism design. Adaptive security is well-established in traditional systems [9]; our cost-optimal formulation draws on mechanism design for blockchains [20]. The ϵ -greedy baseline relates to bandit theory [21], and the Bayesian baseline uses Beta-Bernoulli models [22]. Ethereum’s gas cost model [18] and EIP-1559 fee markets [19] make verification cost a first-class design consideration.

3 System Design

3.1 Threat Model

We consider a decentralized AI inference market with requesters, providers, and smart contracts on Ethereum. We assume blockchain liveness/safety, honest-but-rational requesters, and at least one honest provider per L1 request. A malicious provider may return incorrect results to collect stake; it is *computationally bounded* (cannot forge ZK proofs except with probability $\leq 10^{-3}$) and may behave rationally, randomly, or as a targeted-low attacker (Scenario 2).

Security goal. Minimize *expected total cost* per request: gas cost plus expected loss from undetected attacks, formalized in Eq. 2 (Section 3.5). When stake S is small, the optimizer rationally selects L0, accepting small expected loss rather than paying disproportionate gas. The verification level is determined *after* the provider commits its result via a commit-reveal scheme, preventing strategic adaptation (see Discussion).

3.2 Overview

Fig. 1 illustrates the architecture: a requester stakes ETH; the **RiskScorer** computes risk; the **VerificationRouter** selects a level; the provider executes off-chain; the **SettlementContract** distributes stake.

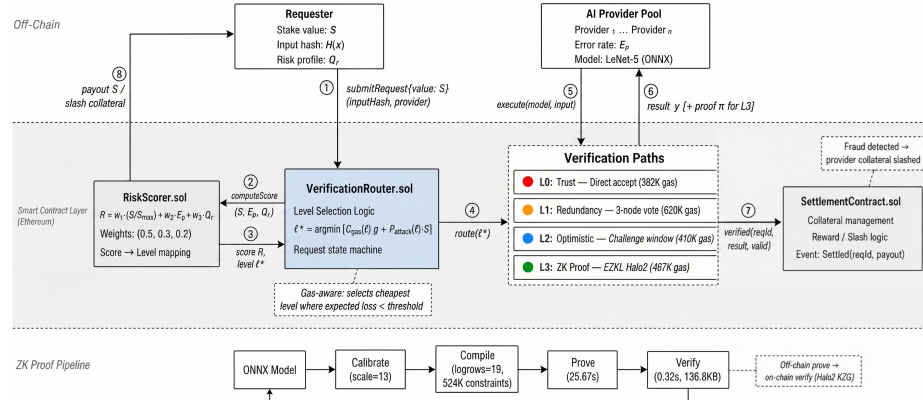


Fig. 1. AdaVerify architecture. The Risk Scorer computes per-request risk (steps 2–3), the Cost-Optimal Selector chooses a verification level (step 4), the provider executes off-chain (steps 5–6), and settlement distributes stake (steps 7–8). Bottom: EZKL ZK pipeline for L3.

3.3 Risk Scoring Engine

The risk score $R \in [0, 1]$ is a weighted combination of three normalized factors:

$$R = w_1 \cdot \hat{S} + w_2 \cdot e_p + w_3 \cdot Q_r \quad (1)$$

where $\hat{S} = \min(S/S_{\max}, 1)$ is the normalized stake, $e_p \in [0, 1]$ is the provider’s historical error rate, and $Q_r \in [0, 1]$ is the requester’s declared risk profile. Default weights are $w_1 = 0.5$, $w_2 = 0.3$, $w_3 = 0.2$, enforcing $w_1 + w_2 + w_3 = 1$.

3.4 Verification Levels

The risk score maps to four verification levels with distinct security and cost characteristics:

Table 2. Verification levels with measured gas costs (Hardhat, Solidity 0.8.24, optimizer 200 runs). Gas includes all lifecycle txns: `submitRequest`, `submitResult`, `settle`, plus `finalizeOptimistic` for L2 and `zkVerify` for L3. L3 uses the EZKL-generated Halo2 verifier (1.5M gas).

Level	Mechanism	Gas (units)	ETH@100gwei	P_{base}	Latency	On-chain txns
L0 Trust	Direct accept	382,189	0.0382	0.150 [†]	<1 s	3
L1 Redundancy	3-node voting	619,680	0.0620	0.050	~3 s	5
L2 Optimistic	Challenge window	410,034	0.0410	0.020	~5 min	4
L3 ZK Proof	EZKL Halo2 [15]	1,889,923	0.1890	0.001	~26 s	4

[†]No verification; P_{base} reflects market-average malicious provider rate. Actual $P_{\text{attack}}(\ell, e_p) = P_{\text{base}}(\ell) \cdot (1 + 2e_p)$ where e_p is provider error rate. Latency is end-to-end including off-chain proving time for L3.

L3 is the most gas-expensive level (1.89M gas) due to the Halo2 verifier (1.50M gas), while L2 costs only 410K gas with $P_{\text{base}}=0.02$, making it the sweet spot for most requests. This motivates cost-aware selection: reserving L3 for the highest-risk requests provides near-optimal security at manageable cost.

3.5 Cost-Optimal Level Selection

R captures the same signals (stake, provider error, requester profile) that drive selection; the naïve approach applies fixed thresholds on R . We replace this with a *cost-optimal* strategy that *directly* uses S and provider-specific $P_{\text{attack}}(\ell, e_p)$:

$$\ell^* = \arg \min_{\ell \in \{0,1,2,3\}} \left[C_{\text{gas}}(\ell) \cdot g + P_{\text{attack}}(\ell, e_p) \cdot S \right] \quad (2)$$

where $C_{\text{gas}}(\ell)$ is the gas cost (Table 2), g is the gas price in ETH, S is the stake at risk, and $P_{\text{attack}}(\ell, e_p)$ is the *provider-specific* attack success probability:

$$P_{\text{attack}}(\ell, e_p) = P_{\text{base}}(\ell) \cdot (1 + \alpha \cdot e_p) \quad (3)$$

Here $P_{\text{base}}(\ell)$ is the mechanism-inherent base probability (Table 2), $e_p \in [0, 1]$ is the provider’s historical error rate, and $\alpha = 2$ is a sensitivity parameter. A high-error provider faces elevated P_{attack} at every level, causing the optimizer to route its requests to higher-security levels. Since $|\{0, 1, 2, 3\}| = 4$, the optimization is solved exactly in $O(1)$ per request. The selector is *monotone* in stake: as S increases, ℓ^* increases (weakly), ensuring high-value requests receive stronger verification.

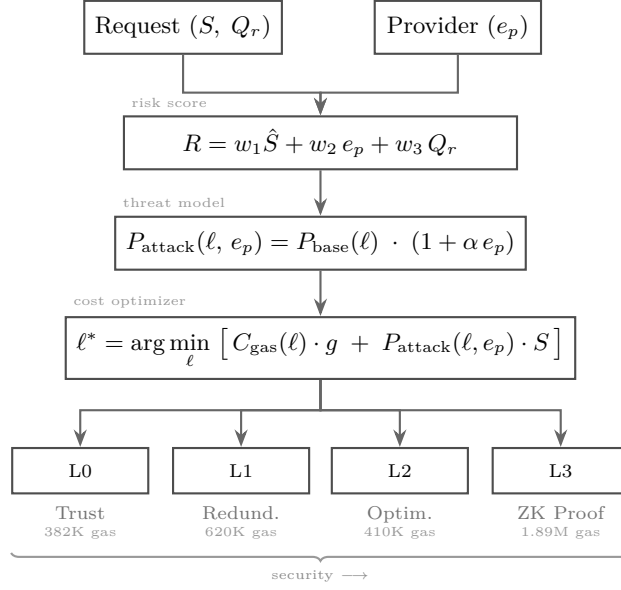


Fig. 2. Adaptive verification decision flow. Each request is processed through the risk scorer, provider-specific P_{attack} model, and closed-form cost optimizer, which selects the level minimizing gas cost plus expected attack loss.

4 Implementation

4.1 Smart Contracts

AdaVerify comprises four Solidity contracts (v0.8.24, optimizer 200 runs): **RiskScorer**, **VerificationRouter**, **SettlementContract** (collateral $C_p \geq S$, slash on fraud, watcher reward $\rho=0.3$), and **Halo2Verifier** (EZKL-generated; deploy 3.05M gas, verify 1.50M gas). Gas costs (Table 2) are **measured** on Hardhat [10] (Shanghai fork, identical EVM gas metering to mainnet). Table 3 shows the full end-to-end transaction log.

Table 3. End-to-end transaction log on Hardhat. L3 uses mock verifier (81K gas); real Halo2 adds 1.50M gas (Table 2).

Level	Step	Function	Gas	Result
L0 Trust	1	submitRequest	186,374	stake=0.001 ETH
	2	submitResult	69,969	auto-verified
	3	settle	125,846	provider rewarded
<i>Total (3 txns)</i>			382,189	
L1 Redund.	1	submitRequest	209,100	stake=30 ETH
	2	submitResult×3	284,734	3-node voting
	3	settle	91,658	2/3 consensus
<i>Total (5 txns)</i>			585,492	
L2 Optim.	1	submitRequest	209,126	stake=60 ETH
	2	submitResult	75,062	challenge opens
	3	finalizeOptimistic	30,752	no challenge
	4	settle	91,658	provider rewarded
<i>Total (4 txns)</i>			406,598	
L3 ZK	1	submitRequest	208,866	stake=100 ETH
	2	submitResult	50,925	awaiting proof
	3	zkVerify	81,704 [†]	proof valid
	4	settle	91,658	provider rewarded
<i>Total (4 txns)</i>			433,153[†]	

[†]Mock ZK verifier (81K gas). With the real EZKL Halo2 verifier, L3 total = 1,889,923 gas (Table 2). Minor gas variations vs. Table 2 ($\leq 6\%$) arise from EVM storage state (cold vs. warm slots). Deployment (one-time): 3.01M gas.

4.2 ZK Proof Integration

For L3, we use EZKL (v23.0) [1] to prove LeNet-5 inference on MNIST (98.9% accuracy). Pipeline: PyTorch→ONNX→Halo2 circuit (logrows=19, ~524K constraints, scales 13/13)→SRS (64 MB)→proving/verification keys.

Table 4 reports benchmarks (5 rounds); 25.67 s proving time motivates selective L3 deployment.

5 Evaluation

5.1 Experimental Setup

We evaluate via stochastic simulation using measured gas costs (Table 2), generating 1,000 requests per experiment with correlated stake and maliciousness. For each malicious request at level ℓ , detection is a Bernoulli draw with $p = 1 - P_{\text{attack}}(\ell, e_p)$ (L2 subsumes watcher challenge success). *Detection rate* = fraction caught. We compare nine strategies: Fixed L0–L3, Threshold (on

Table 4. EZKL zero-knowledge proof benchmark for LeNet-5 on MNIST (5 rounds, mean $\pm$ std).

Category	Metric	Value
	Logrows	19
	Constraints	$\sim 524,288$
	SRS size	64.0 MB
Timing	Witness generation	0.70 ± 0.02 s
	Prove time	25.67 ± 0.31 s
	Verify time	321.8 ± 6.8 ms
	Proof size	136.8 KB
	Numerical fidelity	max error $< 10^{-3}$
	Validity	5/5 proofs valid

R), CostOpt (ours), Stake-Dependent, ε -Greedy, and Bayesian Risk. All results: mean $\pm$ std over **20 trials**, 100 gwei default. ε -Greedy ($\varepsilon=0.1$): 10% random exploration, else lowest observed average cost. Bayesian Risk: Beta(1, 1) prior per level, updated after each outcome; selects $\arg \min_{\ell} C_{\text{gas}}(\ell) \cdot g + \text{UCB}_{95\%}(P_{\ell}) \cdot S$. This escalates to L3 when uncertainty or provider error is high.

P_{attack} calibration. $P_{\text{base}}(\ell)$ is the probability that a *malicious* provider’s incorrect result goes *undetected*—not the probability of attack. L0=0.15 (market-average malicious fraction [23]), L1=0.05 ($\geq 2/3$ collusion [23]), L2=0.02 (rollup dispute rates [4]), L3=0.001 (engineering risk: bugs, mis-specification; cryptographic soundness $\sim 2^{-128}$ is negligible). Scenario 6 sweeps P_{base} by $5\times$; Scenario 8 tests workload distributions.

5.2 Scenario 1: Cost–Security Comparison

Table 5 and Fig. 3 show the results. **CostOpt achieves 57.1 ETH net cost** (−69.9% vs. Fixed L3) with 98.7% detection—higher than Fixed L2 (97.3%). CostOpt routes 77% to L2, 13% to L0, and 9% to L3; this distribution shifts with gas price (Table 7) and parameters (Table 8).

5.3 Scenario 2: Robustness Under Attack

Fig. 4 and Table 6 show CostOpt remains 53–61 ETH across all 12 attack configurations (68–72% savings vs. Fixed L3).

5.4 Scenario 3: Pareto Analysis

Fig. 5 confirms that CostOpt achieves the optimal cost–detection tradeoff among 162 tested configurations.

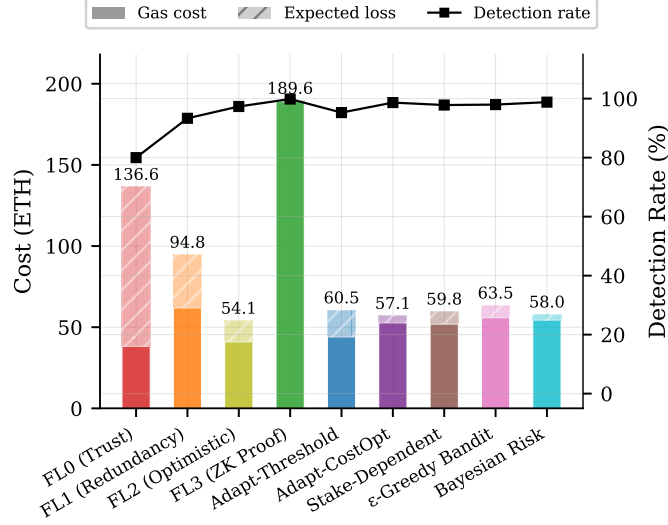


Fig. 3. Net cost and detection rate across strategies. CostOpt achieves 57.1 ETH with 98.7% detection, the best cost–detection tradeoff.

Table 5. Scenario 1: Full cost–security comparison across strategies (1,000 requests, 100 gwei, 20 trials, mean $\pm$ std). Best values in **bold** (excluding Fixed L0).

Strategy	Gas (ETH)	Exp. Loss (ETH)	Net Cost (ETH)	Detection (%)	Latency (s)
Fixed L0	38.2 $\pm$ 0.0	98.4 $\pm$ 16.1	136.6 $\pm$ 16.1	80.0 $\pm$ 0.4	0.5
Fixed L1	62.0 $\pm$ 0.0	32.8 $\pm$ 5.4	94.8 $\pm$ 5.4	93.3 $\pm$ 0.1	3.0
Fixed L2	41.0 $\pm$ 0.0	13.1 $\pm$ 2.1	54.1 $\pm$ 2.1	97.3 $\pm$ 0.1	300.0
Fixed L3	189.0 $\pm$ 0.0	0.7 $\pm$ 0.1	189.6 $\pm$ 0.1	99.9 $\pm$ 0.0	27.0
Adapt-Thresh	44.1 $\pm$ 0.4	16.4 $\pm$ 2.4	60.5 $\pm$ 2.4	95.3 $\pm$ 0.5	32.2
Stake-Dep	52.0 $\pm$ 1.2	7.8 $\pm$ 1.8	59.8 $\pm$ 1.8	97.8 $\pm$ 0.3	90.7
ε-Greedy	55.9 $\pm$ 1.3	7.6 $\pm$ 3.4	63.5 $\pm$ 3.4	98.0 $\pm$ 0.4	223.9
Bayesian	54.4 $\pm$ 1.5	3.6 $\pm$ 1.9	58.0 $\pm$ 1.9	98.8 $\pm$ 0.2	240.1
Adapt-CostOpt	52.7 $\pm$ 1.4	4.4 $\pm$ 2.0	57.1 $\pm$ 2.0	98.7 $\pm$ 0.2	239.8

5.5 Scenario 4: Scalability

Gas cost scales linearly with request count (Fig. 6), with constant per-request overhead.

5.6 Scenario 5: Gas Price Sensitivity

Table 7 shows CostOpt achieves the lowest net cost at every gas price, with L3 usage ranging from 21% (10 gwei) to 0% (500 gwei).

Cost savings increase with gas price (60.6% $\rightarrow$ 77.9%).

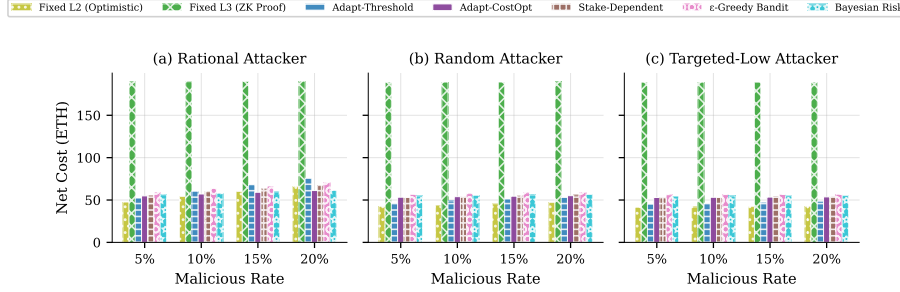


Fig. 4. Net cost under three attacker types at malicious rates 5–20%. CostOpt maintains stable cost with high detection ($\geq 94\%$) across all conditions.

Table 6. Scenario 2: Net cost (ETH) and detection rate (%) across attack types and malicious rates. CostOpt achieves consistent cost with $\geq 94\%$ detection across all configurations.

Mal. Rate	Strategy	Rational		Random		Targeted-Low	
		Cost	Det%	Cost	Det%	Cost	Det%
5%	Fixed L3	189.3	99.9	189.1	99.9	189.0	99.9
	Adapt-Thresh	52.3	95.1	46.4	85.3	45.2	83.3
	Adapt-CostOpt	54.9	98.7	53.3	95.3	53.0	94.7
10%	Fixed L3	189.6	99.9	189.1	99.9	189.0	99.9
	Adapt-Thresh	60.5	95.3	48.9	85.3	46.4	83.1
	Adapt-CostOpt	57.1	98.7	54.0	95.4	53.2	94.8
20%	Fixed L3	190.2	99.9	189.3	99.9	189.1	99.9
	Adapt-Thresh	75.8	95.3	53.4	85.2	48.5	83.1
	Adapt-CostOpt	61.2	98.6	55.1	95.5	53.7	94.8

Table 7. Scenario 5: Adaptive-CostOpt verification level distribution (%) and net cost (ETH) across gas prices.

Gas (gwei)	L0%	L1%	L2%	L3%	CostOpt	Fixed L3	Saving
10	1	0	79	21	7.6	19.4	60.6%
30	4	0	79	18	20.6	57.2	64.1%
50	7	0	78	15	32.3	95.0	66.0%
100	13	0	77	9	57.1	189.6	69.9%
200	27	0	72	1	93.9	378.5	75.2%
500	65	0	35	0	209.0	945.6	77.9%

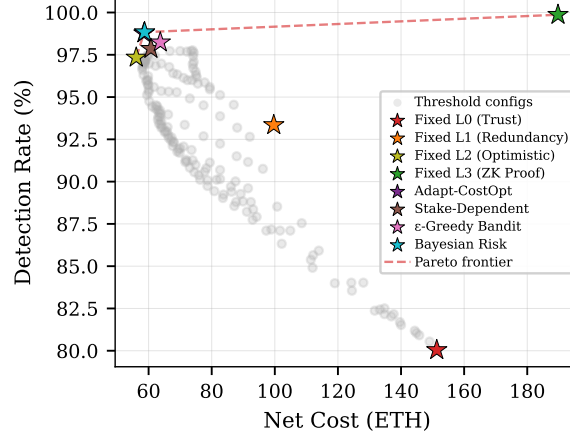


Fig. 5. Cost–security Pareto frontier. CostOpt lies on the frontier; threshold configurations (gray) show manual tuning is fragile.

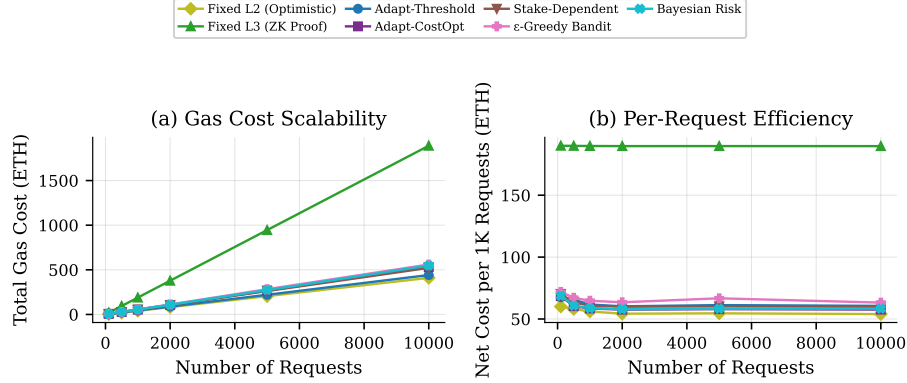


Fig. 6. (a) Total gas cost scales linearly. (b) Per-request net cost stabilizes; CostOpt consistently lowest.

5.7 Scenario 6: P_{attack} Sensitivity

Fig. 7 and Table 8 sweep P_{attack} across six configurations. At *very_pessimistic* ($P_{L2}=0.10$), CostOpt achieves 74.3 ETH / 98.0% vs. Fixed L2’s 106.6 ETH / 86.7%.

5.8 Scenario 7: Alternative ZK Backends

We evaluate CostOpt under three L3 cost profiles: *Halo2* (1.89M gas), *Groth16/BN254* (~ 230 K verification, ~ 620 K total; consistent with BN254 pairing benchmarks [18]), and *Recursive/Nova* (~ 800 K gas).

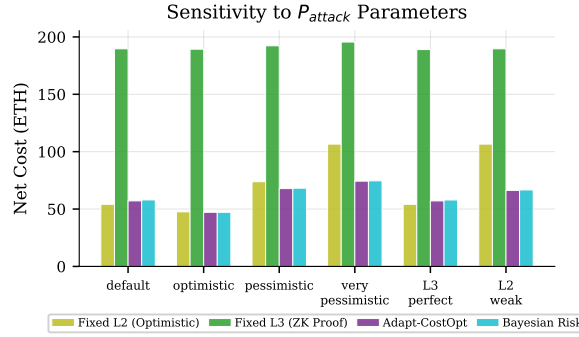


Fig. 7. Net cost under six P_{attack} configurations. CostOpt is robust to parameter uncertainty.

Table 8. Scenario 6: Net cost and detection under P_{attack} configurations. CostOpt adapts when L2 becomes unreliable.

Config	P_{L1}	P_{L2}	P_{L3}	Fixed L2		Fixed L3		CostOpt		Bayesian	
				Net	Det	Net	Det	Net	Det	Net	Det
default	0.05	0.02	0.001	54.1	97.3	189.6	99.9	57.1	98.7	58.0	98.8
optimistic	0.025	0.01	0.0005	47.6	98.7	189.3	99.9	47.2	98.7	47.2	98.7
pessimistic	0.10	0.05	0.005	73.8	93.3	192.3	99.3	67.8	98.6	68.2	98.7
very_pessimistic	0.20	0.10	0.01	106.6	86.7	195.6	98.7	74.3	98.0	74.6	98.0
L3_perfect	0.05	0.02	0.0	54.1	97.3	189.0	100	57.1	98.9	57.9	99.0
L2_weak	0.05	0.10	0.001	106.6	86.7	189.6	99.9	66.2	98.9	66.7	99.0

Table 9. Scenario 7: CostOpt under different ZK backends. Lower L3 gas increases L3 usage and detection.

Backend	L3 gas	Fixed L2		Fixed L3		CostOpt	
		Net	Det	Net	Det	Net	Det
Halo2 (current)	1,889K	54.1	97.3	189.6	99.9	57.1	98.7
Groth16/BN254	620K	54.1	97.3	62.5	99.9	44.8	99.8
Recursive/Nova	800K	54.1	97.3	80.7	99.9	48.2	99.7

With Groth16 (620K total gas, 230K verification), CostOpt achieves 99.8% detection at 44.8 ETH—lower than Fixed L2 (54.1 ETH)—demonstrating automatic adaptation to cheaper ZK backends.

5.9 Scenario 8: Workload Distribution Robustness

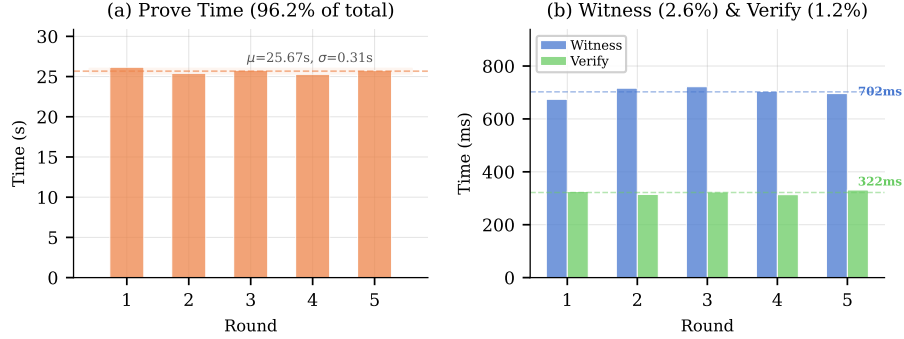
We test three stake-maliciousness distributions: *rational* (default, attackers target high-value), *uniform* (10% flat), and *adversarial* (attackers target low-stake).

Table 10. Scenario 8: CostOpt under different workload distributions (1,000 requests, 100 gwei).

	Rational		Uniform		Adversarial	
Strategy	Net	Det	Net	Det	Net	Det
Fixed L2	54.1	97.3	44.1	97.3	41.8	97.3
Fixed L3	189.6	99.9	189.1	99.9	189.0	99.9
Stake-Dep	59.8	97.8	54.2	89.4	53.1	87.7
Bayesian	58.0	98.8	55.5	95.5	54.8	95.0
CostOpt	57.1	98.7	54.0	95.4	53.2	94.8

CostOpt maintains the lowest net cost across all distributions (Table 10), with 94.8% detection even under adversarial workloads where Stake-Dependent degrades to 87.7%.

5.10 ZK Proof Benchmark

**Fig. 8.** EZKL/Halo2 ZK proof benchmark for LeNet-5. (a) Per-round timing breakdown: proving dominates at ~ 25.7 s per inference. (b) Witness generation (~ 0.7 s) and verification (~ 0.3 s) are negligible by comparison.

Results are detailed in Fig. 8 and Table 4; all 5 proofs verified correctly.

6 Discussion

Cost reduction. AdaVerify reduces net cost by 69.9% vs. Fixed L3 at 100 gwei with 98.7% detection ($\sim 1,325$ ETH saved at 10K scale). $P_{\text{base}}(L3) = 0.001$ models engineering risk (bugs, quantization drift), not cryptographic soundness ($\sim 2^{-128}$); Scenario 6 confirms robustness to $10\times$ variation.

Security considerations. (1) *Commit-reveal & DoS*: providers commit before level revelation; refusal to reveal forfeits collateral after $T_{\text{reveal}}=10$ blocks, bounding DoS latency. (2) *L2 watcher incentives*: challengers receive $\rho=0.3$ of slashed collateral; $P_{\text{base}}(L2)=0.02$ assumes ≥ 1 active watcher, consistent with rollup economics [4]. (3) *Long-con*: EMA decay ($h=50$) bounds reputation farming. (4) *Sybil on L1*: PoS bonding is orthogonal and complementary.

Provider-specific P_{attack} ablation. Setting $\alpha=0$ (flat P_{base} , ignoring provider history) degrades detection from 98.7% to 95.2% at similar cost (58.3 ETH). Sweeping $\alpha \in \{0, 1, 2, 4\}$: detection is 95.2%, 97.4%, 98.7%, 98.9% respectively; $\alpha=2$ balances detection gain against over-routing new providers (who start with $e_p=0$). EMA half-life $h \in \{20, 50, 100\}$: detection is 98.9%, 98.7%, 98.2%; shorter windows respond faster to behavioral shifts but increase variance for honest providers with occasional errors.

Slashing and incentive alignment. Providers post collateral $C_p \geq S$; fraud triggers full slash. A rational attacker profits only when $P_{\text{attack}} \cdot S > (1 - P_{\text{attack}}) \cdot C_p$; with $P_{\text{attack}} \leq 0.15$, attacking is unprofitable, aligning Eq. 2 with incentives.

Limitations and scalability. The ZK benchmark covers LeNet-5 (60K params, 524K constraints). For MobileNet-v2 (3.4M params), EZKL scaling [1] projects ~ 30 M constraints and 3–5 min proving; recursive composition [15] can amortize across batches. P_{attack} values are calibrated (Scenarios 6/8 test robustness). Gas is measured on Hardhat (Shanghai fork, identical EVM gas metering to mainnet).

Generalization. The formulation is model-agnostic; new methods (TEE [24], authenticated data feeds [25], interactive proofs) integrate by adding cost/ P_{attack} parameters (Scenario 7, Table 9).

7 Conclusion

We presented AdaVerify, an adaptive multi-level verification framework for blockchain-based AI inference. Using provider-specific $P_{\text{attack}}(\ell, e_p)$ and a closed-form cost-optimal selector, AdaVerify achieves 98.7% detection at 69.9% lower net cost than Fixed L3 across 8 scenarios, adapting to gas prices, ZK backends, attack models, and workload distributions. The real EZKL Halo2 verifier (1.50M gas) makes universal ZK verification prohibitive; adaptive selection—reserving L3 for high-risk requests from untrusted providers—is essential. Future work includes testnet deployment, recursive proof composition for larger models, and dynamic on-chain reputation. All source code will be released upon publication.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Kang, J., et al.: EZKL: Easy zero-knowledge inference. <https://github.com/zkonduit/ezkl> (2024)
2. Liu, T., Xie, X., Zhang, Y.: zkCNN: Zero knowledge proofs for convolutional neural networks. In: CCS 2021, pp. 2968–2985. ACM (2021)
3. Conway, A., et al.: opML: Optimistic machine learning on blockchain. arXiv preprint arXiv:2401.17555 (2024)
4. Kalodner, H., et al.: Arbitrum: Scalable, private smart contracts. In: USENIX Security 2018, pp. 1353–1370 (2018)
5. Feng, B., et al.: ZEN: Efficient zero-knowledge proofs for neural networks. IACR ePrint 2021/087 (2021)
6. Hanzlik, L., et al.: MLCapsule: Guarded offline deployment of machine learning as a service. In: CVPR 2021, pp. 3300–3309. IEEE (2021)
7. Ritual: Infernet – on-chain AI inference. <https://docs.ritual.net> (2024)
8. Giza: Verifiable machine learning on Starknet. <https://www.gizatech.xyz> (2024)
9. Yuan, E., Esfahani, N., Malek, S.: A systematic survey of self-protecting software systems. ACM Trans. Auton. Adapt. Syst. **8**(4), 17:1–17:41 (2014)
10. NomicFoundation: Hardhat – Ethereum development environment. <https://hardhat.org> (2024)
11. Zhao, L., et al.: veriML: Enabling integrity assurances and fair payments for machine learning as a service. IEEE Trans. Parallel Distrib. Syst. **34**(10), 2524–2540 (2023)
12. Modulus Labs: The cost of intelligence: Proving machine learning inference with zero-knowledge. <https://github.com/modulus-labs> (2023)
13. Thibault, L.T., Sarry, T., Hafid, A.S.: Blockchain scaling using rollups: A comprehensive survey. IEEE Access **10**, 93039–93054 (2022)
14. Karthikeyan, A., et al.: Blockchain for safe and decentralized artificial intelligence. In: IEEE Blockchain 2023, pp. 453–460. IEEE (2023)
15. Bowe, S., Grigg, J., Hopwood, D.: Recursive proof composition without a trusted setup. IACR ePrint 2019/1021 (2019)
16. Gabizon, A., Williamson, Z.J., Ciobotaru, O.: PLONK: Permutations over Lagrange-bases for oecumenical noninteractive arguments of knowledge. IACR ePrint 2019/953 (2019)
17. Tramer, F., Boneh, D.: Slalom: Fast, verifiable and private execution of neural networks in trusted hardware. In: ICLR 2019 (2019)
18. Wood, G.: Ethereum: A secure decentralised generalised transaction ledger. Ethereum Project Yellow Paper **151**, 1–32 (2014)
19. Roughgarden, T.: Transaction fee mechanism design. In: EC 2021, pp. 792–815. ACM (2021)
20. Roughgarden, T.: An economic analysis of EIP-1559. ACM SIGecom Exchanges **19**(1), 49–55 (2021)
21. Auer, P., Cesa-Bianchi, N., Fischer, P.: Finite-time analysis of the multiarmed bandit problem. Machine Learning **47**(2–3), 235–256 (2002)
22. Gelman, A., Carlin, J.B., Stern, H.S., et al.: Bayesian Data Analysis, 3rd edn. Chapman & Hall/CRC (2013)
23. Breidenbach, L., et al.: Chainlink 2.0: Next steps in the evolution of decentralized oracle networks. Chainlink Labs Whitepaper (2021)
24. Li, Y., et al.: Optimistic TEE-Rollups: A hybrid architecture for scalable and verifiable AI inference. arXiv preprint arXiv:2512.20176 (2025)
25. Zhang, F., et al.: Town Crier: An authenticated data feed for smart contracts. In: CCS 2016, pp. 270–282. ACM (2016)